Disassembly/translation/assembly procedure Breed for Xiaomi Mi Router 3G with NAND flash memory.

A couple of years ago I read a post **dabyd64** on the OpenWrt forum post Success on hacking, extracting, modifying (Translating) and repacking BREED Bootloader!, it described the procedure for disassembling/translating/assembling Breed for SPI flash memory and just one patch.

For more sophisticated bootloaders, the process is somewhat more complex – getting ahead – calculating three checksums using two different algorithms, etc.
I'll also note that the CRC specified as POSIX in article by **johovich** on Habr. If you use, for example, `pycrc` with the `--model=posix` switch (https://pycrc.org/) or soltau's CRC Calculator (https://sourceforge.net/projects/crc-calculator/), the checksum will likely be calculated correctly, but this is apparently not the same «POSIX CRC» used in Breed!

Special thanks to **Nicky F.** from the 4PDA forum for his responsiveness and for the Python script for disassembling and assembling Breed!
I didn't really understand much of the script, but that's because I'm not a programmer, not because the script is incorrect!

Link to **Breed 1.2 r1416 [2022-07-24) (git-46ae2a1)** for Xiaomi Mi Router 3G
breed-mt7621-xiaomi-r3g.bin (MD5: e65d388129a6d1ac39abf99329f1978b)
https://breed.hackpascal.net/r1416%20%5b2022-07-24%5d/breed-mt7621-xiaomi-r3g.bin
https://breed.hackpascal.net/breed-mt7621-xiaomi-r3g.bin

1. Tools

A) binwalk Firmware Analysis Tool
https://github.com/ReFirmLabs/binwalk
B) u-boot-tools. More precisely dumpimage from the u-boot-tools package
C) XZ Utils. More precisely lzmainfo from XZ Utils
https://github.com/tukaani-project/xz
D) lzma from LZMA SDK. It has been experimentally established that lzma-data in Breed are packed lzma version 4.62
https://sourceforge.net/projects/sevenzip/files/LZMA%20SDK/4.62/
E) Hex editor. I used 010 Editor
https://www.sweetscape.com/010editor/
F) HTTP server. I used HTTP File Server (HFS)
https://github.com/rejetto/hfs
G) *NIX utilities: dd, cat, crc32, cksum, printf. Optional date.
All utilities are present in the port BusyBox for Windows
https://frippery.org/busybox/
H) PuTTY
Breed's author writes about the incompatibility between Breed and the Linux telnet client, and suggests using PuTTY: «*Please use the telnet client that comes with Windows or PuTTY. The telnet client under Linux is not compatible*».
https://putty.software/
I) Windows + A virtual machine with Linux to run binwalk and dumpimage, or just Linux.

2. Examples of command usage. In Windows, everything is the same as in Linux, just precede the command with busybox

Example of command input crc32 in Windows

```
busybox crc32 04NoHeader-MiR3G-Cn
```

Conversion from decimal to hexadecimal

```
printf %X 1658592583
62DC1D47
```

Conversion from hexadecimal to decimal

```
printf %d 0x62DC1D47
1658592583
```

Calculates the CRC32 value. The output is a hexadecimal number.

```
crc32 04NoHeader-MiR3G-Cn
df0a279a 04NoHeader-MiR3G-Cn
```

POSIX CRC32/cksum calculation. Output is a decimal number.

```
cksum 01uImage-Header-MiR3G-Cn-Mod1
26846098 64 01uImage-Header-MiR3G-Cn-Mod1
```

Conversion from decimal to hexadecimal

```
printf %X 26846098
199A392
```

Converting a date/time to «Unix Epoch» decimal

```
date -d "2022-07-23 16:09:43" +%s
1658592583
```

Converting a «Unix Epoch» Date to a Date/Time

```
date -R -u @1658592583
Sat, 23 Jul 2022 16:09:43 +0000
```

Convert the current (at the time the command is executed) date/time to a decimal «Unix Epoch» number

```
date -R -u +%s
```

Cut the first 64 bytes from breed-mt7621-xiaomi-r3g.bin and save it to the file 01uImage-Header-MiR3G-Cn

```
dd if=breed-mt7621-xiaomi-r3g.bin of=01uImage-Header-MiR3G-Cn bs=1 count=64 skip=0
```

Indent the first 64 bytes in breed-mt7621-xiaomi-r3g.bin, cut out a piece of 28156 bytes and save it to the file 02Boot-MiR3G-Cn

```
dd if=breed-mt7621-xiaomi-r3g.bin of=02Boot-MiR3G-Cn bs=1 count=28156 skip=64
```

Step back 28220 bytes from the beginning of the file and save the remaining portion to the file 03Data MiR3G-Cn.lzma

```
dd if=breed-mt7621-xiaomi-r3g.bin of=03Data-MiR3G-Cn.lzma bs=1 skip=28220
```

Consecutively merge the files 01uImage-Header-MiR3G-En, 02Boot-MiR3G-En, 03Data-MiR3G-En.lzma and write them to the file breed-mt7621-xiaomi-r3g-En-Test.bin

```
cat 01uImage-Header-MiR3G-En 02Boot-MiR3G-En 03Data-MiR3G-En.lzma >
breed-mt7621-xiaomi-r3g-En-Test.bin
```

In Windows, you can use the copy command with the `/b` switch

```
copy /b 01uImage-Header-MiR3G-En + 02Boot-MiR3G-En + 03Data-MiR3G-En.lzma
breed-mt7621-xiaomi-r3g-En-Test.bin
```

3. Launch binwalk, analyze the Breed file

```
binwalk breed-mt7621-xiaomi-r3g.bin

DECIMAL   HEXADECIMAL   DESCRIPTION
--------------------------------------------------------------------------------
0         0x0           uImage header, header size: 64 bytes,
                        Header CRC: 0x2CD9C16A, created: 2022-07-23 16:09:43,
                        image size: 137186 bytes, Data Address: 0xA0201000,
                        Entry Point: 0xA0201000, Data CRC: 0xDF0A279A,
                        OS: Linux, CPU: MIPS, image type: Standalone Program,
                        compression type: none, image name: "Breed MT7621"
24213     0x5E95        JBOOT STAG header, image id: 7, timestamp 0x5004418,
                        image size: 1074814976 bytes, image JBOOT checksum: 0x218,
                        header JBOOT checksum: 0x2100
27552     0x6BA0        Copyright string: "Copyright (C) 2022 HackPascal
                        <hackpascal@gmail.com>"
28220     0x6E3C        LZMA compressed data, properties: 0x6D,
                        dictionary size: 33554432 bytes,
                        uncompressed size: 373234 bytes
```

4. Cutting the Breed file

```
dd if=breed-mt7621-xiaomi-r3g.bin of=01uImage-Header-MiR3G-Cn bs=1 count=64 skip=0
dd if=breed-mt7621-xiaomi-r3g.bin of=02Boot-MiR3G-Cn bs=1 count=28156 skip=64
dd if=breed-mt7621-xiaomi-r3g.bin of=03Data-MiR3G-Cn.lzma bs=1 skip=28220
```

5. Check compression settings. Note down the Dictionary size, values of lc, lp and pb

```
lzmainfo.exe 03Data-MiR3G-Cn.lzma
Uncompressed size:             0 MB (373234 bytes)
Dictionary size:               32 MB (2^25 bytes)
Literal context bits (lc):     1
Literal pos bits (lp):         2
Number of pos bits (pb):       2
```

6. Unpacking data in Chinese

```
lzma d 03Data-MiR3G-Cn.lzma 03Data-MiR3G-Cn
```

7. Editing Data. The Nuances of Breed Translation

Copy/rename the data file from 03Data-MiR3G-Cn to 03Data-MiR3G-En
Open 03Data-MiR3G-En in a hex editor and set the display encoding to UTF-8.
It is highly advisable to turn on the Highlighting of control characters.
In 010 Editor:
View → Character Set → UTF-8
View → Highlighting → Control Characters, Zeros

The first words in Chinese are next to the line «`BREED:ABORT`», just below.
In Unicode, each Chinese character in UTF-8 encoding takes up 3 bytes (4 bytes for rare characters; I don't remember that's I've seen 4-bit characters in Breed).
«Chinese Unicode» also includes punctuation marks: spaces, periods, commas, exclamation marks, colons, etc.
And each Chinese punctuation mark also takes up 3 (three) bits!
We're replacing the characters with more understandable symbols. We're doing everything slowly, carefully, and attentively!

7.1. The file size (in bytes) must not change! Keep an eye on this.
Create backups regularly.

7.2. It is necessary to replace, but under no circumstances touch the control characters `Null [00], HT [09], LF [0A], CR [0D]` etc.
All control characters must remain in their values and places!

7.3. Do not edit lines that do not contain hieroglyphs. Without disassembling, it is not known for certain what they mean in the Breed code. Damaging them can lead to Breed not working, which is what is called bricking the router!

7.4. Replace only hieroglyphs with 7-bit ASCII characters (the Unicode "Basic Latin" block) – you can't go wrong!

7.5. If space is needed, one additional character can be obtained by changing the HTML tag containing the space. For example: `<br /> → <br/>`

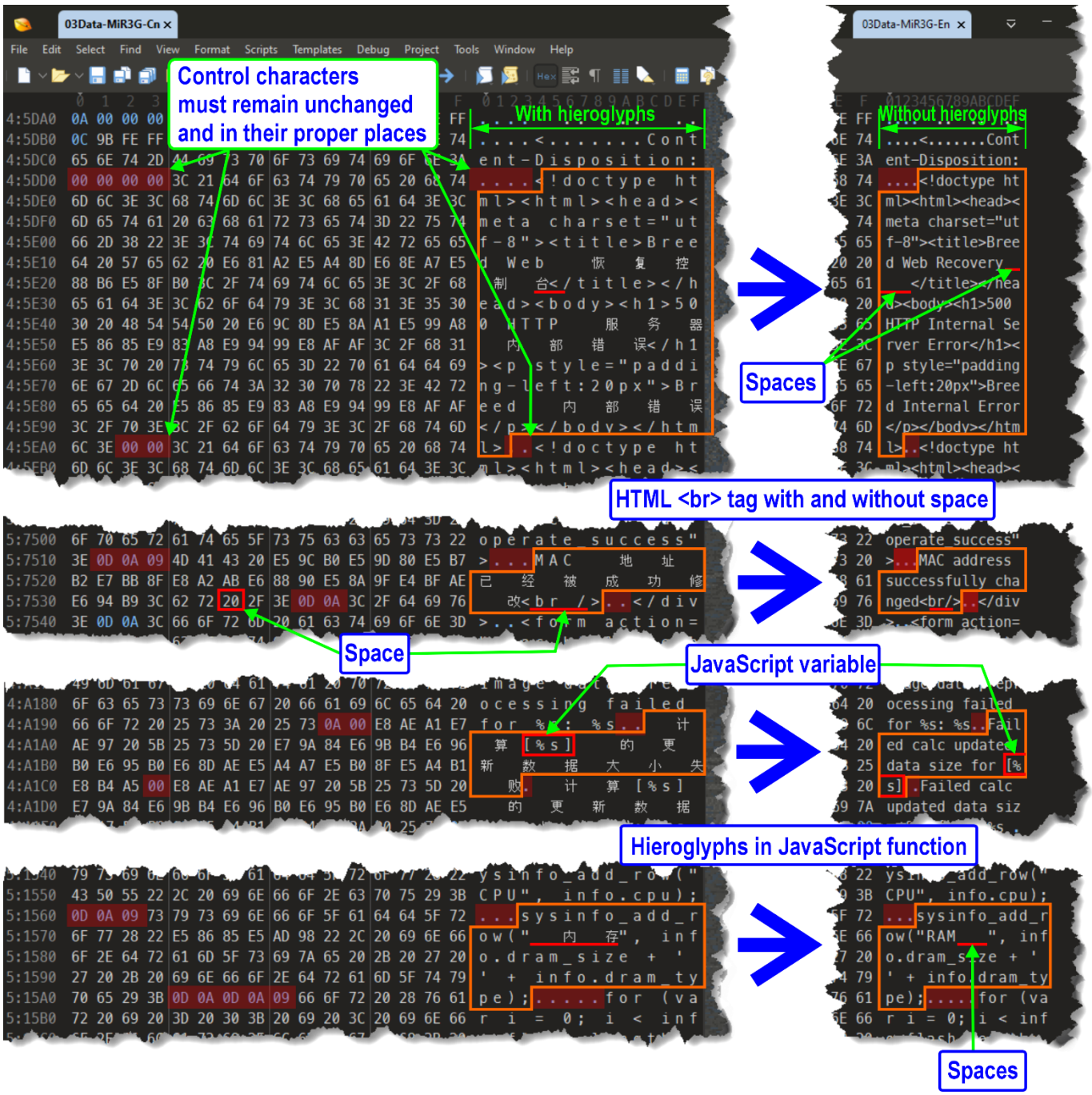
7.6. If there is no space on the right, but there are spaces on the left within HTML tags bordering control characters, then shift the HTML tags and translation strings to the left.
All HTML tags must be preserved!

7.7. If, on the contrary, there are hieroglyphs/one or two bits of a hieroglyph left to the right of the translated text, then they should be replaced with spaces.

7.8. Be careful when editing JavaScript functions containing hieroglyphs and strings containing JavaScript variables in addition to hieroglyphs.

7.9. If the translation doesn't insert, shorten it and find synonyms. It will still be much clearer than the hieroglyphs!

7.10. In 010 Editor, the width of the window where editing is corrected and contains Chinese characters is approximately one and a half times wider than without them. A decrease in width can be considered an indicator that the translation process is complete.



8. Repack the data in English. Use lzma 4.62

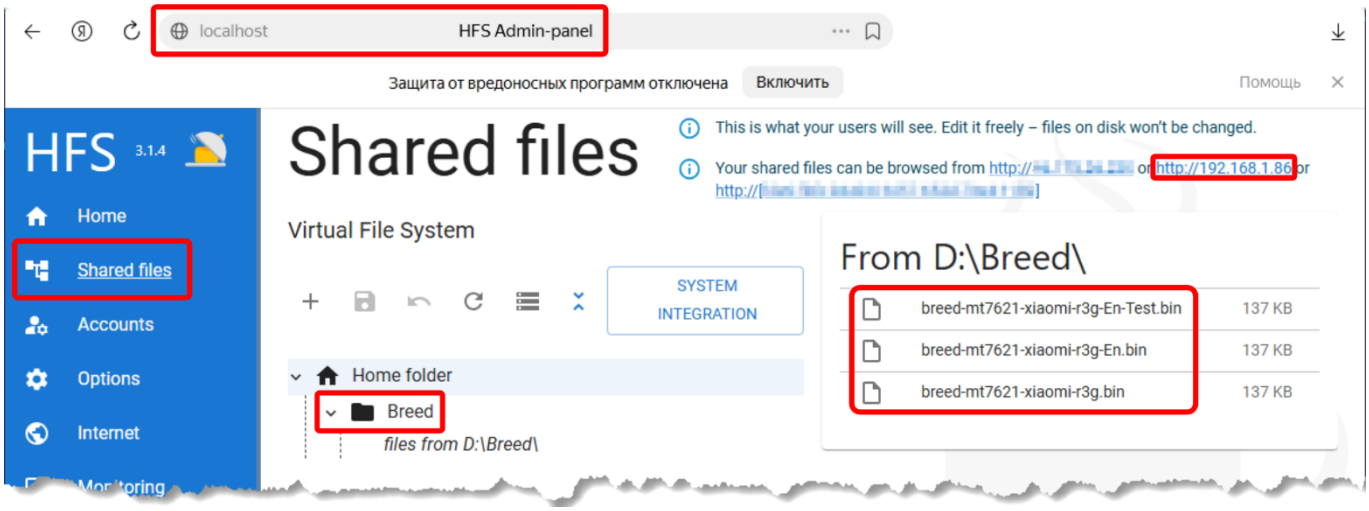
```
lzma e 03Data-MiR3G-En 03Data-MiR3G-En.lzma -d25 -lc1 -lp2 -pb2
```

9. Intermediate Breed build for testing without writing to the router's flash memory

```
cat 01uImage-Header-MiR3G-Cn 02Boot-MiR3G-Cn 03Data-MiR3G-En.lzma >
breed-mt7621-xiaomi-r3g-En-Test.bin
or
copy /b 01uImage-Header-MiR3G-Cn + 02Boot-MiR3G-Cn + 03Data-MiR3G-En.lzma
breed-mt7621-xiaomi-r3g-En-Test.bin
```

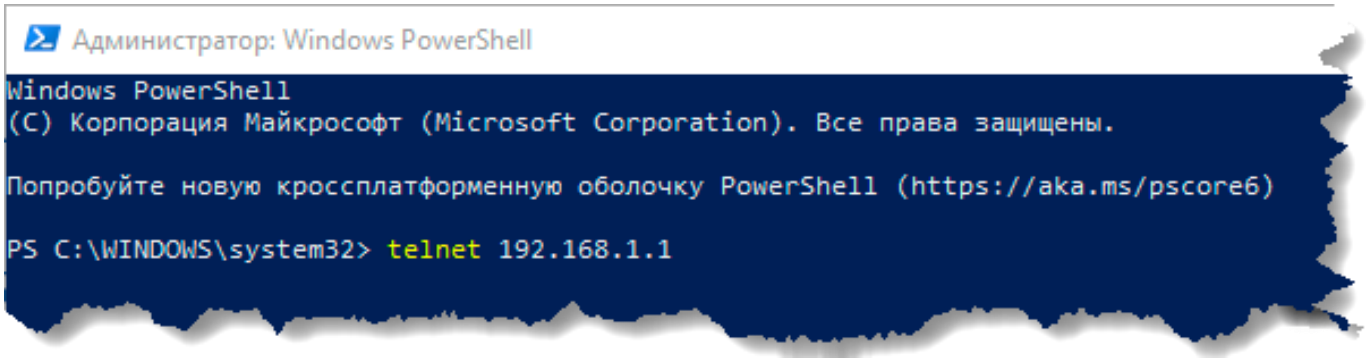
10. Testing, checking the result

10.1. Download HTTP File Server (HFS).
10.2. Switch off Wi-Fi.
10.3. Launch HFS. Admin panel → Shared files. Create a folder in which files will be accessible via HTTP.
In my example, the folder is named Breed, IP-address 192.168.1.86, file name of Breed – breed-mt7621-xiaomi-r3g-En-Test.bin



10.4. Turn off the router and turn it on, holding the Reset button, wait 7-10 seconds after the LEDs start blinking, release Reset.
10.5. Connecting the computer to the router with a patch cord.
10.6. Open your browser in «Incognito» mode and go to 192.168.1.1 in the Breed Web Interface.
10.7. Switch to PowerShell. Connect to the router via Telnet. Enter the following command in PowerShell:

```
telnet 192.168.1.1
```

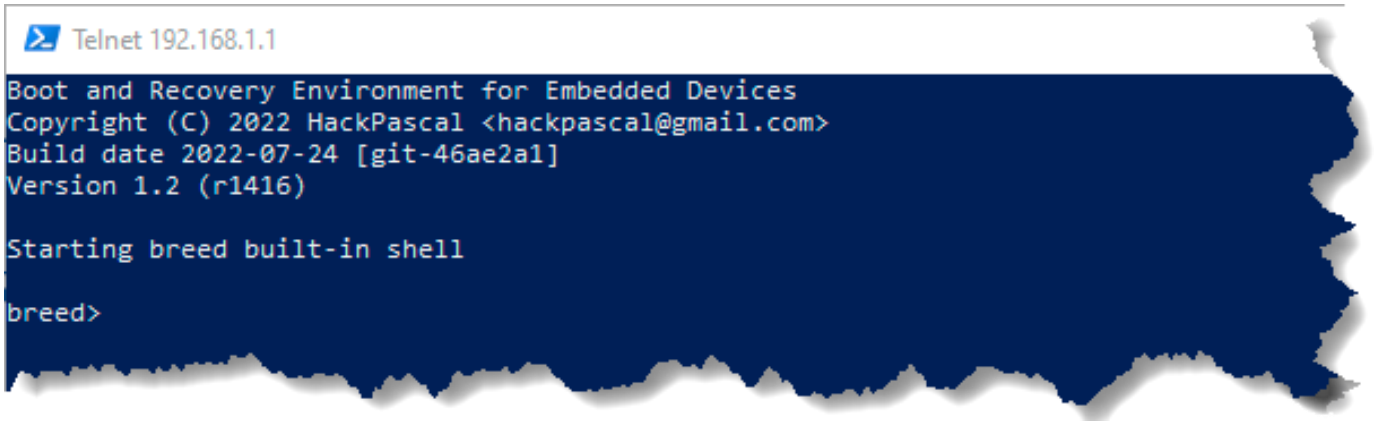


10.8. PowerShell Terminal output:

```
Boot and Recovery Environment for Embedded Devices
Copyright (C) 2022 HackPascal <hackpascal@gmail.com>
Build date 2022-07-24 [git-46ae2a1]
Version 1.2 (r1416)

Starting breed built-in shell

breed>
```



10.9. Enter the command in Breed:

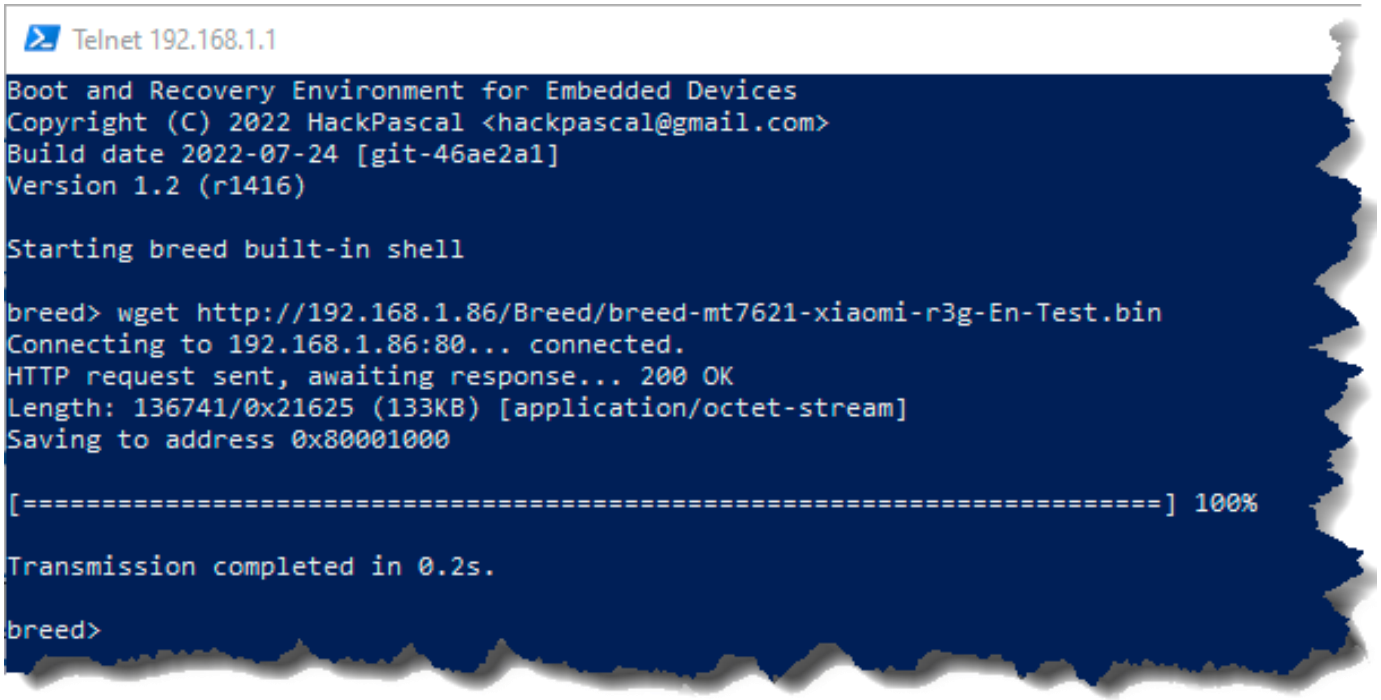
```
wget http://192.168.1.86/Breed/breed-mt7621-xiaomi-r3g-En-Test.bin
```

PowerShell Terminal output:

```
Connecting to 192.168.1.86:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 136741/0x21625 (133KB) [application/octet-stream]
Saving to address 0x80001000

[============================================================================] 100%

Transmission completed in 0.2s.
```

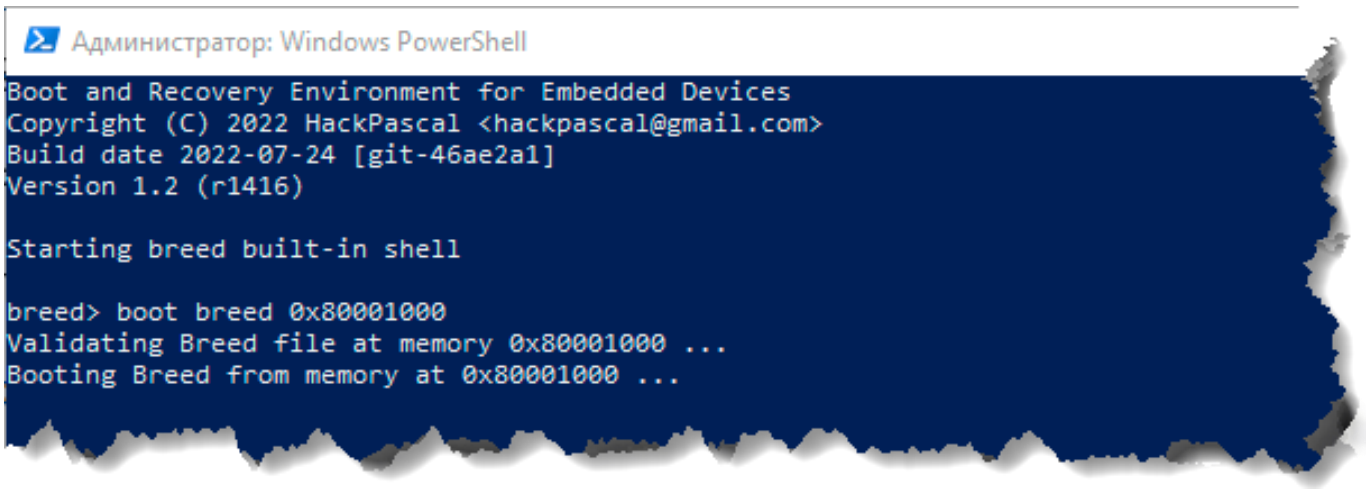


10.10. Next, enter the command in Breed:

```
boot breed 0x80001000
```

PowerShell Terminal output:

```
Validating Breed file at memory 0x80001000 ...
Booting Breed from memory at 0x80001000 ...
```



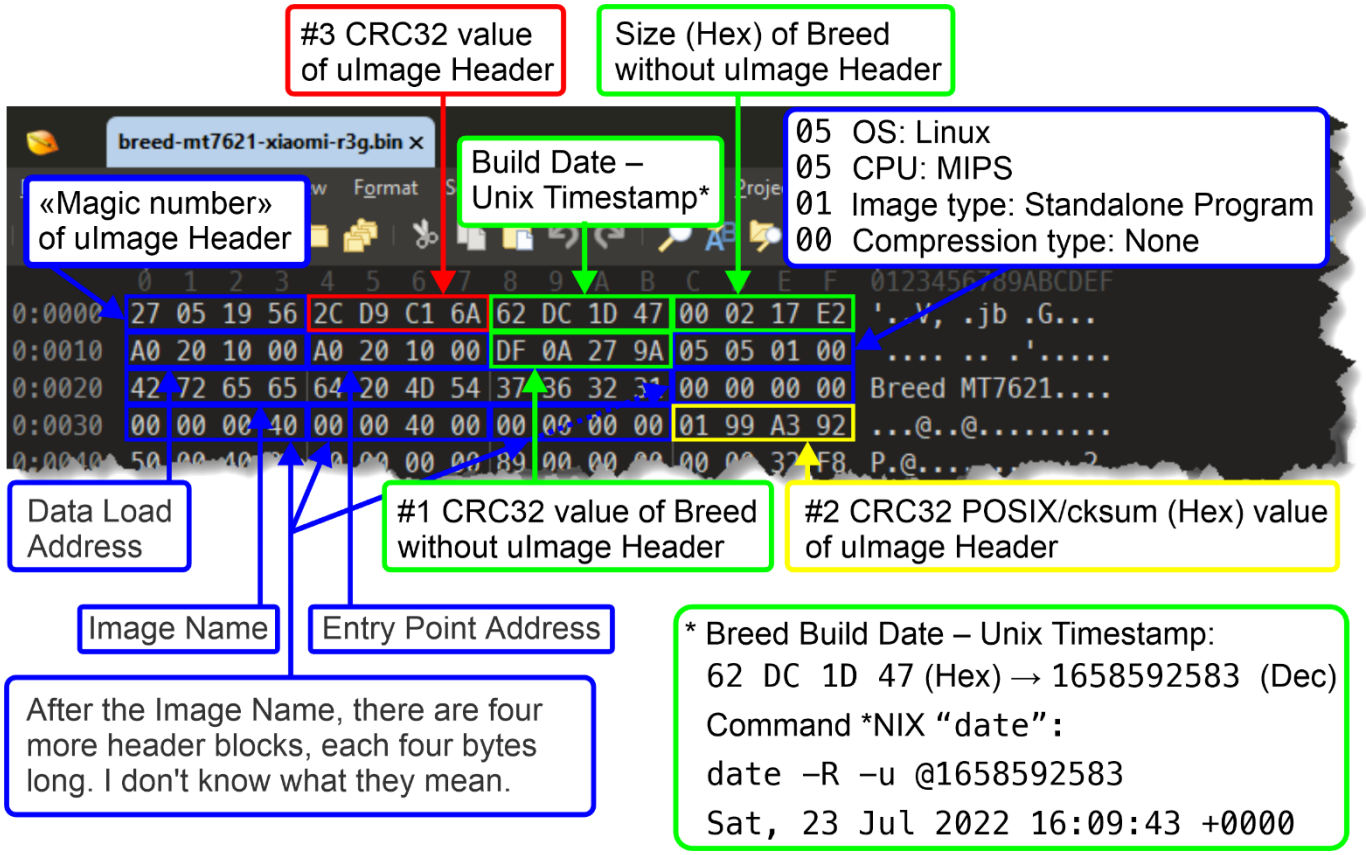
10.11. Switch to your browser and refresh the page (press F5).
The translated Breed should appear.

10.12. Review the results of your work, checking all menus, tables, checkboxes, etc. for functionality, the presence/absence of errors, translation accuracy, etc.
Do not attempt to flash breed-mt7621-xiaomi-r3g-En-Test.bin into the router's Flash-memory. It's useless. You'll receive an error message in Chinese, translated, something like this: «Unable define flash layout of [Bootloader] specify manually» or «Data preprocess. [Bootloader] failed: Invalid data».

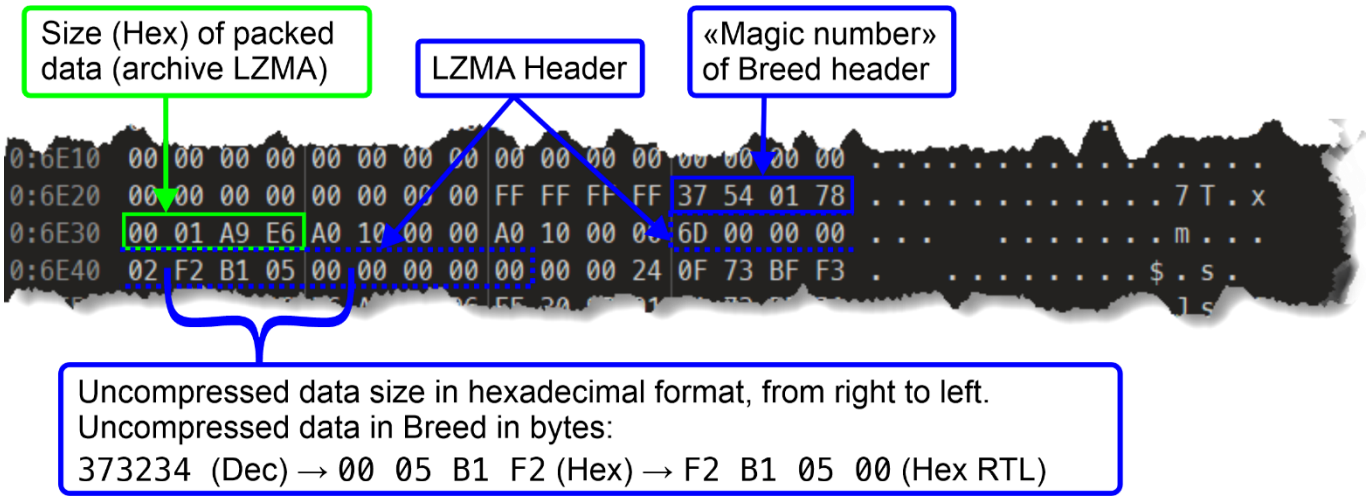
10.13. If you find typos/errors, etc., edit/pack it again 03Data-MiR3G-En, build again breed-mt7621-xiaomi-r3g-En-Test.bin, again
`wget/boot breed 0x80001000 ...`
If everything is ok, let's move on to the most interesting part!

11. A few words about headers in the Breed Bootloader

uImage Header. Breed 1.2 r1416 [2022-07-24) (git-46ae2a1)
breed-mt7621-xiaomi-r3g.bin
(MD5: e65d388129a6d1ac39abf99329f1978b)



Breed 1.2 r1416 [2022-07-24) (git-46ae2a1) Header
breed-mt7621-xiaomi-r3g.bin
(MD5: e65d388129a6d1ac39abf99329f1978b)



12. Forming uImage Header

12.1. Look at the size of the final 03Data-MiR3G-En.lzma, note down it.

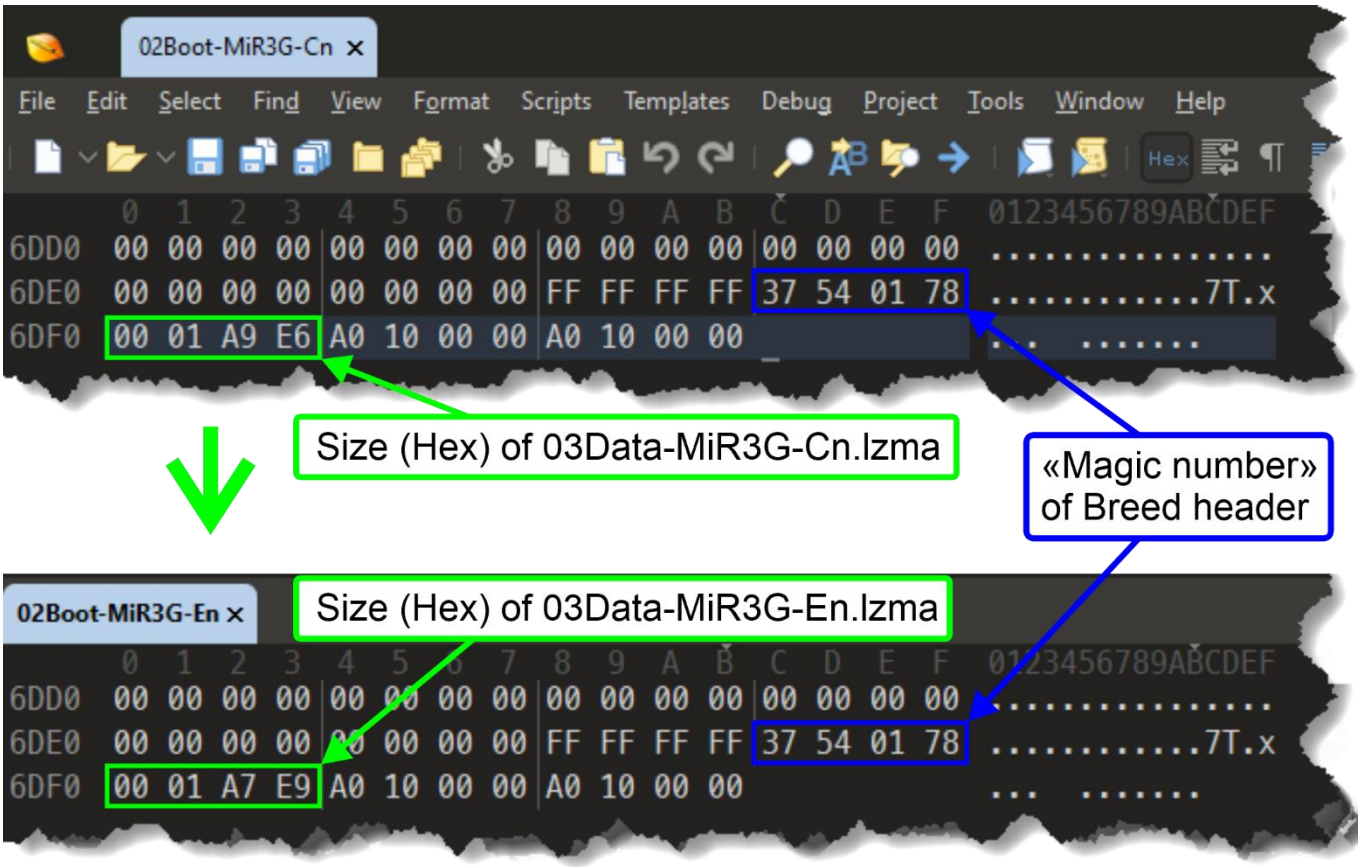
12.2. Converting the size of 03Data-MiR3G-En.lzma from decimal to hexadecimal:

```
printf %X 108521
1A7E9
```

12.3. Open 02Boot-MiR3G-Cn in a hex editor.
Find the Breed header by its hexadecimal value (the «Magic number» of the Breed header) **37 54 01 78** the next 4 bytes are the size of 03Data-MiR3G-Cn.lzma in hexadecimal: **00 01 A9 E6**

12.4. Replace **00 01 A9 E6** with **00 01 A7 E9**.

```
0000006DE0: 00 00 00 00 00 00 00 00 FF FF FF FF 37 54 01 78
0000006DF0: 00 01 A9 E6 A0 10 00 00 A0 10 00 00
            ↓  ↓  ↓  ↓
0000006DE0: 00 00 00 00 00 00 00 00 FF FF FF FF 37 54 01 78
0000006DF0: 00 01 A7 E9 A0 10 00 00 A0 10 00 00
```



12.5. Save the file as 02Boot-MiR3G-En.

12.6. Build Breed without uImage Header:

```
cat 02Boot-MiR3G-En 03Data-MiR3G-En.lzma > 04NoHeader-MiR3G-En
```

or

```
copy /b 02Boot-MiR3G-En + 03Data-MiR3G-En.lzma 04NoHeader-MiR3G-En
```

12.7. Looking at the size of 04NoHeader-MiR3G-En, note down it.
In my example, the size of 04NoHeader-MiR3G-En is 136677 bytes.

12.8. Converting the size of 04NoHeader-MiR3G-En from decimal to hexadecimal:

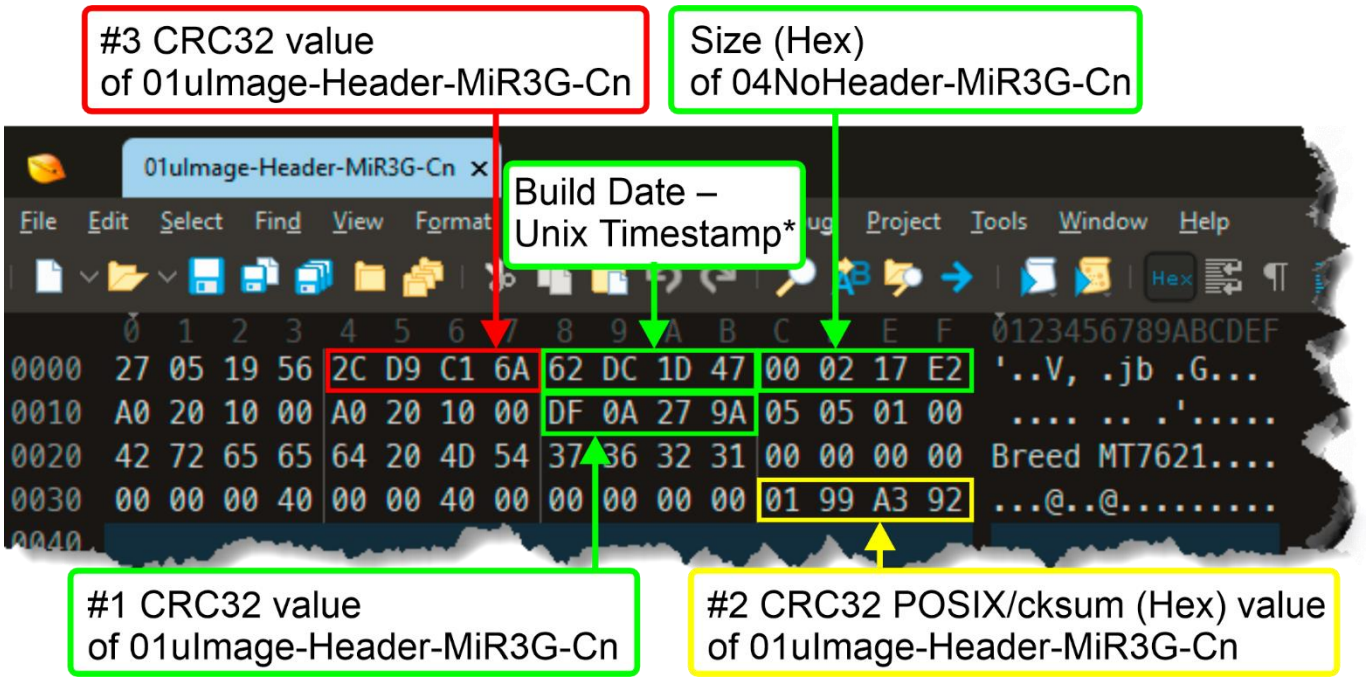
```
printf %X 136677
215E5
```

12.9. Calculate the checksum 04NoHeader-MiR3G-En using the CRC32 algorithm:

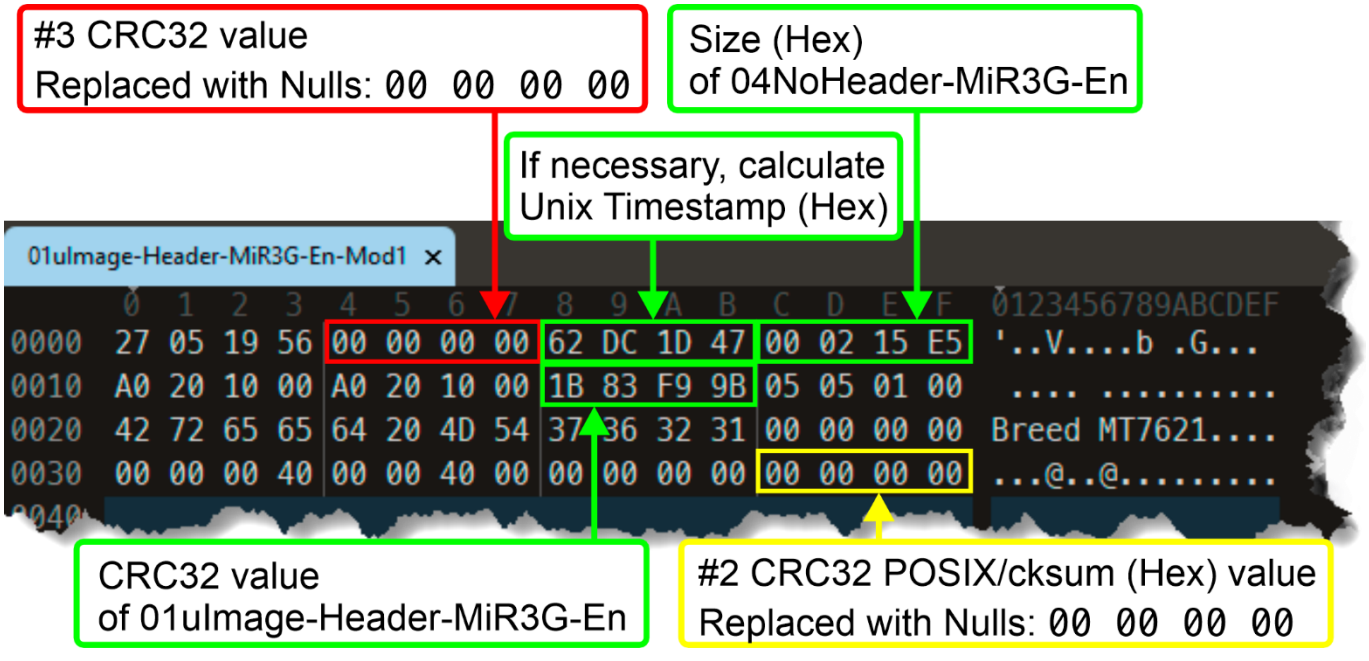
```
crc32 04NoHeader-MiR3G-En
1b83f99b 04NoHeader-MiR3G-En
```

12.10. Open 01uImage-Header-MiR3G-Cn in a hex editor

```
0000000000: 27 05 19 56 2C D9 C1 6A 62 DC 1D 47 00 02 17 E2
0000000010: A0 20 10 00 A0 20 10 00 DF 0A 27 9A 05 05 01 00
0000000020: 42 72 65 65 64 20 4D 54 37 36 32 31 00 00 00 00
0000000030: 00 00 00 40 00 00 40 00 00 00 00 00 01 99 A3 92
```



Bytes 05 through 08 are replaced with Nulls:
`2C D9 C1 6A` → `00 00 00 00`
Bytes 13 through 16 are replaced with size (Hex) of 04NoHeader-MiR3G-En:
`00 02 17 E2` → `00 02 15 E5`
Bytes 25 through 28 are replaced with CRC32 value of 04NoHeader-MiR3G-En:
`DF 0A 27 9A` → `1B 83 F9 9B`
Bytes 61 through 64 are replaced with Nulls:
`01 99 A3 92` → `00 00 00 00`



12.10.1. Optional. I kept the original Timestamp Breed in Chinese.
Bytes 09 through 12 contain the assembly date – Unix Timestamp, if necessary, you can add a Timestamp to the header at this stage.
For the Unix Timestamp calculation algorithm, see Section **2. Examples of command usage**. The date command date
Next, we convert the decimal number «Unix Epoch» to hexadecimal and enter it into the header in bytes 09 through 12.

12.11. Save the file as 01uImage-Header-MiR3G-En-Mod1

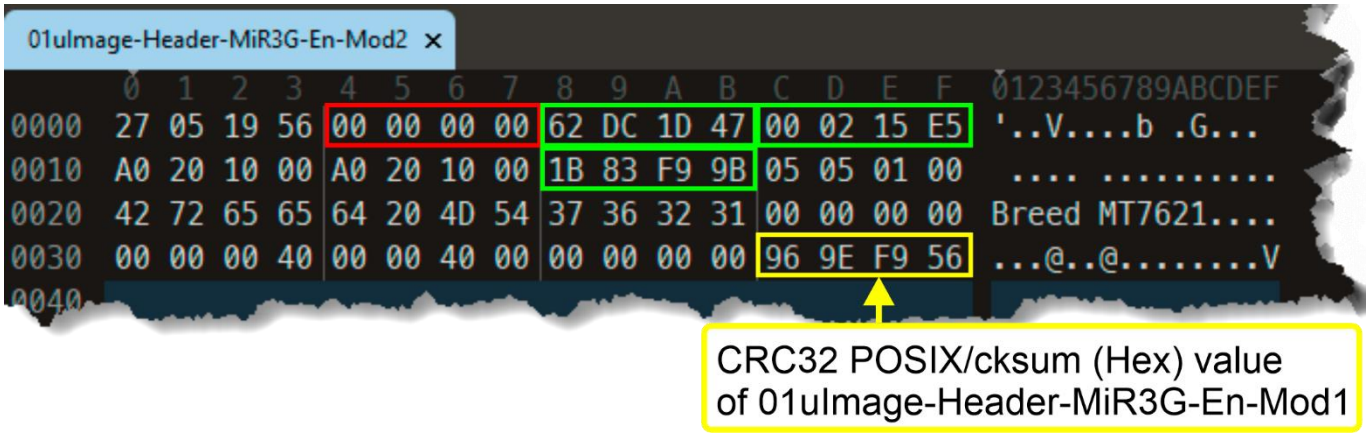
12.12. Calculate the checksum of 01uImage-Header-MiR3G-En-Mod1 using the CRC32 POSIX/cksum algorithm:

```
cksum 01uImage–Header–MiR3G–En–Mod1
2527000918 64 01uImage–Header–MiR3G–En–Mod1
```

12.13. Converting 2527000918 from decimal to hexadecimal:

```
printf %X 2527000918
969EF956
```

12.14. Open 01uImage-Header-MiR3G-En-Mod1 in a hex editor
Bytes 61 through 64 are replaced with
CRC32 POSIX/cksum (Hex) value of 01uImage-Header-MiR3G-En-Mod1:
`00 00 00 00` → `96 9E F9 56`

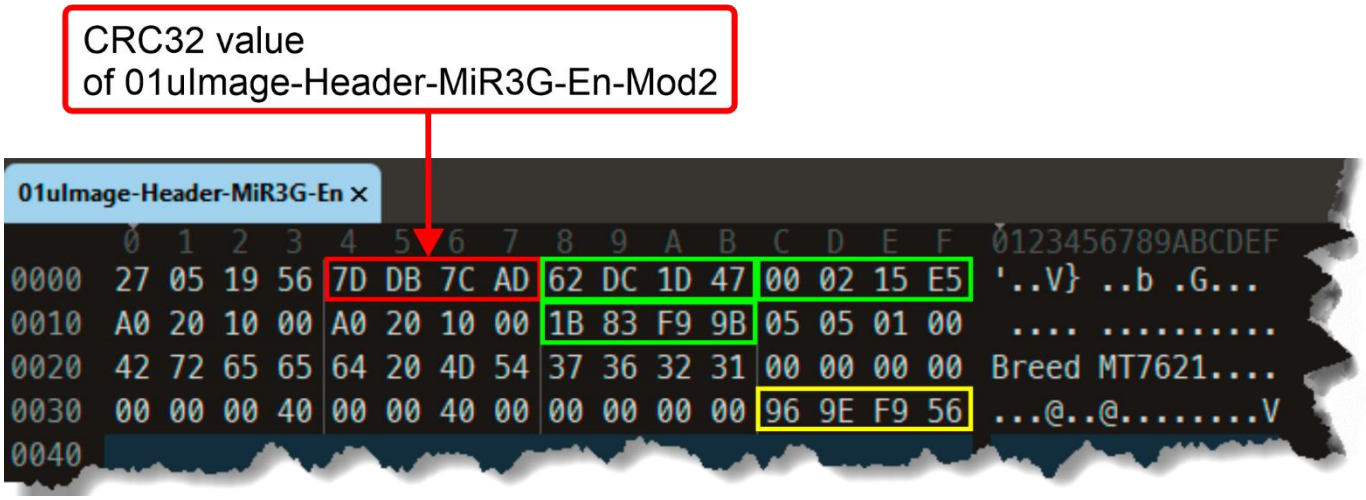


12.15. Save the file as 01uImage-Header-MiR3G-En-Mod2

12.16. Calculate the checksum 01uImage-Header-MiR3G-En-Mod2 using algorithm CRC32:

```
crc32 01uImage-Header-MiR3G-En-Mod2
7ddb7cad
```

12.17. Bytes 05 through 08 are replaced with CRC32 value of 01uImage-Header-MiR3G-En-Mod2:
00 00 00 00 → 7D DB 7C AD



12.1. Save the file as 01uImage-Header-MiR3G-En

13. Final build of the Breed image

```
cat 01uImage-Header-MiR3G-En 04NoHeader-MiR3G-En > breed-mt7621-xiaomi-r3g-En.bin
```

or

```
copy /b 01uImage-Header-MiR3G-En + 04NoHeader-MiR3G-En breed-mt7621-xiaomi-r3g-En.bin
```

14. Checking breed-mt7621-xiaomi-r3g-En.bin

14.1. Check the assembled file of Breed using binwalk. Note the binwalk output values for Header CRC (header bytes 05 to 08) and Data CRC (header bytes 25 to 28).

```
binwalk breed-mt7621-xiaomi-r3g-En.bin

DECIMAL   HEXADECIMAL   DESCRIPTION
--------------------------------------------------------------------------------
0         0x0           uImage header, header size: 64 bytes, Header CRC: 0x7DDB7CAD,
                        created: 2022-07-23 16:09:43, image size: 136677 bytes,
                        Data Address: 0xA0201000, Entry Point: 0xA0201000,
                        Data CRC: 0x1B83F99B, OS: Linux, CPU: MIPS,
                        image type: Standalone Program,
                        compression type: none, image name: "Breed MT7621"
24213     0x5E95        JBOOT STAG header, image id: 7, timestamp 0x5004418,
                        image size: 1074814976 bytes, image JBOOT checksum: 0x218,
                        header JBOOT checksum: 0x2100
27552     0x6BA0        Copyright string: "Copyright (C) 2022 HackPascal
                        <hackpascal@gmail.com>"
28220     0x6E3C        LZMA compressed data, properties: 0x6D,
                        dictionary size: 33554432 bytes,
                        uncompressed size: 373234 bytes
```

14.2. Checking the assembled file of Breed using dumpimage -l.
If the output is like this:

```
dumpimage -l breed-mt7621-xiaomi-r3g-En.bin
Image Name:   Breed MT7621
Created:      Sat Jul 23 12:09:43 2022
Image Type:   MIPS Linux Standalone Program (uncompressed)
Data Size:    136677 Bytes = 133.47 KiB = 0.13 MiB
Load Address: a0201000
Entry Point:  a0201000
```

Everything is almost 100% good!

If the output is like this:

```
dumpimage -l breed-mt7621-xiaomi-r3g-En.bin
Truncated file
dumpimage: cannot detect image type
```

There's probably a typo/mistake in the size/CRC calculation chain.
We'll double-check everything!

14.3. For added emphasis, we'll partially re-run Section **10. Testing, checking the results**, and if no errors or typos are found and Breed displays a message when attempting to flash the Bootloader without red hieroglyphs and a plate containing the Latin letters MD5 at the bottom left, then you're ready to flash!

15. Useful links

15.1. **Success on hacking, extracting, modifying (Translating) and repacking BREED Bootloader!**
https://forum.openwrt.org/t/success-on-hacking-extracting-modifying-translating-and-repacking-breed-bootloader/137710

15.2. **U-Boot modification for MT7621 Devices**
https://github.com/pinney/MT7621-u-boot-mod/tree/master

15.3. **Breed bootloader breed-mt7621-xiaomi-r3g.bin reset button patcher**
https://github.com/legale/breed-mt7621-xiaomi-r3g.bin-reset-button-changer

15.4. **Binwalk – Полное руководство по инструменту для извлечения образа прошивки (Binwalk – A Complete Guide to the Firmware Image Extractor Tool)**
https://forensicanvil.ru/forum/topic/tools/binwalk-instrumentu-izvlecheniya

15.5. **Реверс-инжиниринг домашнего роутера с помощью binwalk (Reverse Engineering a Home Router with Binwalk)**
https://habr.com/ru/articles/487406/

15.6. **История одного маленького реверс-инжиниринга или как мы BREED для Beeline Smartbox FLASH/GIGA расковыряли (The story of one small reverse engineering project, or how we uncovered BREED for Beeline Smartbox FLASH/GIGA)**
https://habr.com/ru/articles/649039/

15.7. **Реверс-инжиниринг домашнего роутера с помощью binwalk. Доверяете софту своего роутера? (Reverse engineering a home router using binwalk. Do you trust your router's software?)**
https://habr.com/ru/articles/487406/